Algorithm Analysis and Correctness

Running a program tells us what happened on one execution. Analyzing an algorithm asks more: **Does it return a correct answer for every legal input? How does its cost grow with the input?**

We develop specifications and proofs, exact operation counts, asymptotic comparisons, and loop sums. Insertion sort provides one continuous example, from individual moves to a general cost formula.

Prerequisites: arrays, loops, elementary algebra and logarithms. Area and limit arguments also use basic calculus.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 1–3; summations in Appendix A.

Reading route

1. Specifications and termination → linear-search invariants → a complete insertion-sort analysis.
2. Asymptotic quantifiers → growth comparisons → sums and loop counts.
3. Integer boundaries and space → interpreting costs → mathematical extensions.

Read the main sequence before the extensions; revisit definitions when a symbol is unfamiliar.

Specify the answer before reading the code

A problem has a legal input domain D and an output relation $R(x)$. One input $x \in D$ may admit several answers. An algorithm is a precise procedure for producing one of them.

A sorting input is a sequence of records with a key and a fixed identity. In $(3, a), (1, b), (3, c)$, the letters distinguish records; they are not secondary sorting keys.

Output requirement	How to check it
Nondecreasing keys	Adjacent keys satisfy $A[i].key \leq A[i+1].key$
No lost or duplicated records	The output is a permutation, including multiplicities
Stability, when required	Equal keys retain their original relative order

Sortedness alone is insufficient: deleting every record leaves a sorted empty sequence, but does not sort the original input.

Identities distinguish equal keys

Input [3a, 1b, 3c] → sort by key; letters identify records

Sorted, preserved, stable	Sorted and preserved	Sorted, but missing a record
1b 3a 3c	1b 3c 3a	1b 3a
All three requirements hold	Equal-key a and c are reversed	3c is lost: invalid sorting

Stability is an additional requirement. Swapping equal-key records can preserve sortedness and the input multiset while violating stability. Stability also requires the original order of a and c.

Empty and singleton inputs are already sorted. A correct implementation must handle them without reading a nonexistent first record.

A correct result and termination are separate obligations

Partial correctness means that if execution terminates, its output satisfies the specification. Total correctness additionally requires termination on every legal input.

```
while true
    skip
```

This program never returns an incorrect answer, but never returns an answer at all. A proof about its result after termination does not establish that it completes the task.

For a loop, a common termination argument uses a nonnegative integer that strictly decreases in every iteration. A finite program listing or a finite collection of tests does not replace that argument.

Linear search: specify which occurrence to return

Given $A[1..n]$ and target x, return the **first** index with $A[i] = x$, or NIL if none exists. Allow $n = 0$; one record read or comparison costs a constant.

```
FIRST-MATCH(A, x)
    i = 1
    while i <= n
        if A[i] == x: return i
        i = i + 1
    return NIL
```

Arrays are indexed from 1. An interval $A[a..b]$ with $a > b$ is empty; it does not access an element. Here and in the sorting pseudocode, n denotes the input array length.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Array references are different from record identities

Our pseudocode accesses an array through a reference. Updating $A[i]$ changes the array visible to the caller. Rebinding the procedure's local variable to another array does not rebind the caller's variable.

Operations	Result
$A = [3, 1]$; $B = A$; then $B[1] = 8$	Both names refer to one array; A becomes $[8, 1]$
$A = [3, 1]$; $B = \mathit{copy}(A)$; then $B[1] = 8$	The arrays have separate storage; A stays $[3, 1]$

A shallow array copy may still contain references to the same objects. Independent array storage does not imply a deep copy of every object. The identities a, b, c used in sorting identify records, not necessarily their memory addresses.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The search invariant records what has been ruled out

Before each `i <= n` test: $1 \le i \le n+1$, and no position in $A[1..i-1]$ matches x.

Stage	Reason
Initialization	$i = 1$; the excluded prefix is empty
Maintenance	A failed comparison followed by $i \leftarrow i + 1$ extends that prefix by one
Match found	$A[i] = x$, and every earlier position has been excluded
Loop exhausted	$i = n + 1$; every position has been excluded, so NIL is correct

Every iteration that does not return decreases the nonnegative integer $n - i + 1$ by 1. Thus execution stops. For $n = 0$, it returns NIL immediately without reading $A[1]$.

Before i is excluded; after i is still unknown

A = [6, 2, 6, 5]; target x = 5

	1	2	3	4	
i = 1	6	2	6	5	Excluded: empty Current: no match
i = 3	6	2	6	5	Excluded: A[1..2] Current: no match
i = 4	6	2	6	5	Excluded: A[1..3] Current: match; return 4

Green: excluded. Teal border: current. White: not yet tested.

For $A = [6, 2, 6, 5]$, search for 5. Before $i = 3$, only positions 1–2 are excluded. A failed comparison extends that prefix; at $i = 4$, the first match returns. **Untested does not mean excluded.**

Three complete executions of the same code

Input and target	Positions compared	Return value	Key comparisons
$[4, 2, 4]$, target 4	1	1	1
$[4, 2, 4]$, target 7	1, 2, 3	NIL	3
Empty array, target 4	None	NIL	0

The cases test first occurrence, exhaustion, and empty input; the invariant supplies the general proof.

If the first matching position, conditional on success, is uniform over $1..n$, the expected comparison count conditional on success is $(n + 1)/2$. Another distribution requires another calculation.

State the cost model before measuring cost

Initially use a word-RAM model: indices and records occupy a constant number of machine words; reads, writes, comparisons, and supported basic arithmetic operations have constant cost. This is a model, not a claim that every real operation takes the same time.

Object of analysis	Possible cost unit
Array search	Key comparisons or reads
Sorting	Comparisons, moves, auxiliary space
Large-integer algorithms	Bit lengths and multiword arithmetic
External storage	Block transfers

For integers with b bits, arbitrary-precision multiplication is not automatically one constant-time operation. Read every bound together with its model.

Input length is not necessarily the numerical value

A positive integer N has binary length $b = \lfloor \lg N \rfloor + 1$, where $\lg = \log_2$. A loop with N iterations may be exponential in the input length b.

For example, $N = 13$ is `1101`, only four bits long. Enumerating every integer up to N takes $\Theta(N)$ iterations. Since $2^{b-1} \le N < 2^b$, that is $\Theta(2^b)$ iterations as a function of b.

Array size usually means the number of records n. Graph bounds often need both the vertex count V and edge count E. Writing $O(V + E)$ preserves that distinction.

Choose the cost function: best, worst, or average

For inputs of length n in D_n, let $C(x)$ be the cost on input x.

$$B(n) = \min_{x \in D_n} C(x), \qquad W(n) = \max_{x \in D_n} C(x).$$

Average cost additionally needs a distribution P_n:

$$A(n) = \mathbb{E}_{x \sim P_n}[C(x)] = \sum_{x \in D_n} P_n(x) C(x).$$

Big-O does not mean worst case. Each of B, W, A is a function with its own upper and lower bounds. For a randomized algorithm, also distinguish randomness in the input from the algorithm's own random choices.

Insertion sort extends a sorted prefix

Use $[(7, a), (2, b), (5, c), (2, d), (1, e)]$, abbreviated $[7a, 2b, 5c, 2d, 1e]$. Save the next record as `key` and shift larger records on its left one position right.

```
INSERTION-SORT(A)
    for j = 2 to n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

`and` short-circuits left to right, so $A[i]$ is not read when $i = 0$. For $n \le 1$ the outer loop is empty; strict comparison keeps older equal-key records before the new one.

Every iteration leaves a checkable result

j / saved record	Key comparisons, right to left	Shifts h_j	Array after insertion
2 / $2b$	$7 > 2$; reach boundary	1	$2b, 7a, 5c, 2d, 1e$
3 / $5c$	$7 > 5$; $2 > 5$ is false	1	$2b, 5c, 7a, 2d, 1e$
4 / $2d$	$7 > 2, 5 > 2$; $2 > 2$ is false	2	$2b, 2d, 5c, 7a, 1e$
5 / $1e$	$7, 5, 2d, 2b$ all exceed 1	4	$1e, 2b, 2d, 5c, 7a$

$2b$ remains before $2d$. There are eight shifts but ten key comparisons: iterations 3 and 4 each have a final failed comparison; iterations 2 and 5 reach $i = 0$ without another key comparison.

A logical hole explains temporary duplicates

Fixed j = 4; saved key = 2d

	1	2	3	4	
1	2b	5c	7a	□	7 > 2: shift 7a right
2	2b	5c	□	7a	5 > 2: shift 5c right
3	2b	□	5c	7a	2 > 2 is false: stop shifting
4	2b	2d	5c	7a	Insert key; 2b stays before 2d

□ is a logical hole, not a written value. The saved key remains outside the array.

The box is a **logical hole**, not a written empty marker. The positions outside it plus saved key retain the input. After the first shift, physical memory holds two copies of $7a$; the whole physical prefix is not a permutation.

Outer invariant: order, preservation, and stability

Let S be a snapshot of the original input. Before each iteration, $A[1..j-1]$ is a stable sorting of $S[1..j-1]$, and $A[j..n] = S[j..n]$.

Stage	How the invariant holds
Start, $j = 2$	One record is already sorted; the suffix is unchanged
Maintenance	Shift the larger suffix and insert key; equal keys do not cross; the unprocessed suffix is unchanged
Stop	At $j = n + 1$, the sorted prefix contains the whole original input

The inner index decreases at every shift, with at most $j - 1$ shifts; there are at most $n - 1$ outer iterations. Thus the algorithm terminates. Inputs with $n \leq 1$ are handled separately by the empty loop.

Inner invariant: include the position of the hole

Fix j. Let $B = A[1..j]$ at the start of this iteration, with $B[1..j-1]$ sorted. Save $key = B[j]$ and place the logical hole at $i + 1$.

Before each inner test:

1. $A[1..i] = B[1..i]$.
2. $A[i + 2..j] = B[i + 1..j - 1]$; every key in this right segment exceeds key.
3. Excluding the hole and adding the saved key preserves exactly the records in B.

Initially the right segment is empty. Copying $A[i]$ into the hole and decrementing i preserves all three properties. On stopping, the left segment is at most key and the right segment exceeds it; filling the hole restores a sorted, record-preserving prefix.

Count comparisons, moves, and guard tests separately

For $n \geq 2$, let h_j count shifts in iteration j. Let $q_j = 1$ if it stops at a failed key comparison with $i \geq 1$, and $q_j = 0$ otherwise.

Operation	Iteration j	Entire execution
Shift write	h_j	$\sum_{j=2}^{n} h_j$
Final key write	1	$n-1$
Complete inner guard test	$h_j + 1$	$\sum h_j + n - 1$
Key comparison	$h_j + q_j$	$\sum h_j + \sum q_j$

Here $\sum h_j = 8$ and $\sum q_j = 2$: ten key comparisons, twelve guard tests, and twelve array writes. Reading key into a local variable is not an array write.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why does the shift count equal the inversion count?

An inversion is a pair of original positions $p < q$ with $S[p].key > S[q].key$. Denote the total by $I(S)$.

When inserting the original record $S[j]$, the earlier $j-1$ records are sorted. Exactly those with keys larger than $S[j]$ shift right. Thus h_j counts precisely the inversions whose right endpoint is j.

Every inversion has a unique right endpoint, giving

$$\sum_{j=2}^{n} h_j = I(S), \qquad T(S) = \Theta(n + I(S)).$$

Equal keys do not form inversions. The example has $1, 1, 2, 4$ inversions by right endpoint, totaling eight, exactly as in the shift table.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Best, worst, and average for one algorithm

Input assumption	Inversions	Time
Already nondecreasing	0	$\Theta(n)$
Distinct keys in decreasing order	$n(n-1)/2$	$\Theta(n^2)$
All permutations of distinct keys equally likely	Expected $n(n-1)/4$	Expected $\Theta(n^2)$

For the last row, let X_{pq} indicate whether positions $p < q$ form an inversion. A uniform permutation gives $\mathbb{E}[X_{pq}] = 1/2$. By linearity of expectation,

$$\mathbb{E}[I] = \sum_{p<q} \mathbb{E}[X_{pq}] = \binom{n}{2}\frac{1}{2}.$$

Independence between these indicators is unnecessary. Repeated keys or a nonuniform input model require a new distribution argument.

What asymptotic analysis ignores, and what it retains

We study growth as $n \to \infty$, allowing fixed multiplicative constants and finitely many small inputs to be disregarded for the bound. Small inputs still require correct behavior.

Here n is a positive integer, f is eventually nonnegative, and g is eventually strictly positive. $O(g), \Omega(g), \Theta(g)$ are **sets of functions**. The conventional notation $f = O(g)$ means $f \in O(g)$.

For example, $n \in O(n^2)$ is true but loose. Use Θ when both sides have been established; Big-O alone does not mean an exact growth order.

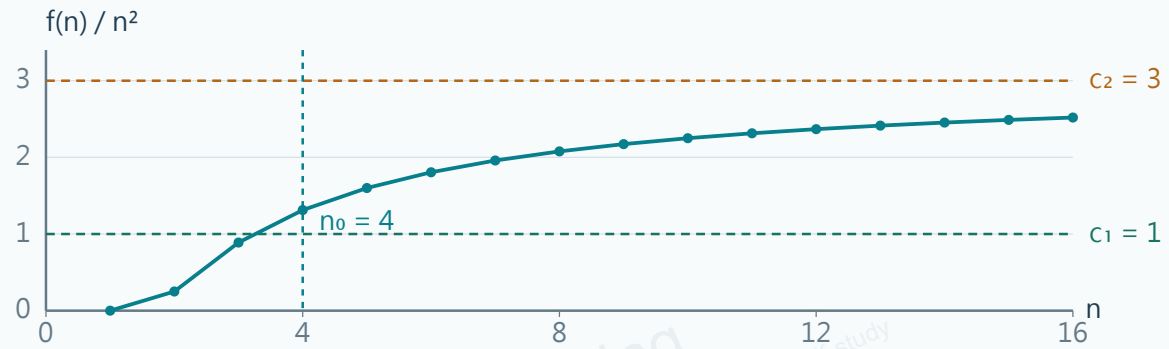
The quantifiers in upper, lower, and tight bounds

Notation	Formal requirement: for every integer $n \geq n_0$
$f \in O(g)$	There exist $c > 0, n_0$ with $0 \leq f(n) \leq cg(n)$
$f \in \Omega(g)$	There exist $c > 0, n_0$ with $0 \leq cg(n) \leq f(n)$
$f \in \Theta(g)$	There exist $c_1, c_2 > 0, n_0$ with $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

Constants may depend on the fixed functions, **not on input size** n. The threshold need not be the smallest possible one.

To refute $f \in O(g)$, every candidate c, n_0 must admit an $n \geq n_0$ with $f(n) > cg(n)$. Defeating only one proposed pair is insufficient.

Plot the constants and the threshold together



For every n ≥ 4, the ratio lies between 1 and 3; earlier values need not.

Let $f(n) = 3n^2 - 8n + 5$. For $n \geq 4$, $f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1$; for $n \geq 1$, $f(n) \leq 3n^2$.

Thus $c_1 = 1, c_2 = 3, n_0 = 4$ prove $f \in \Theta(n^2)$. The plot illustrates the ratio. The inequalities, rather than finitely many plotted points, cover every $n \geq 4$.

Prove that the same function has no linear upper bound

We have $f(n) = 3n^2 - 8n + 5 \geq n^2$ for $n \geq 4$. Suppose $f \in O(n)$. Then some fixed c, n_0 satisfy $f(n) \leq cn$ for every $n \geq n_0$.

Choose an integer $n > \max\{4, n_0, c\}$. It follows that

$$n^2 \leq f(n) \leq cn \quad \Longrightarrow \quad n \leq c,$$

contradicting $n > c$. Therefore $f \notin O(n)$.

The order of choices matters: allow arbitrary candidate constants first, then choose a large enough input. One fixed test input cannot refute every candidate bound.

Use the larger of two thresholds

Suppose (c_U, N_U) witnesses $f \in O(g)$ and (c_L, N_L) witnesses $f \in \Omega(g)$. Both inequalities hold for $n \geq \max(N_U, N_L)$, so

$$\Theta(g) = O(g) \cap \Omega(g).$$

Other familiar rules also have assumptions:

Property	Reason or condition
Transitivity of O	Multiply the constants; take the larger threshold
$f \in O(g) \Leftrightarrow g \in \Omega(f)$	Read the same inequality in reverse with reciprocal constant
$f + g = \Theta(\max(f, g))$	For nonnegative functions, $\max(f, g) \leq f + g \leq 2\max(f, g)$

If cancellation is allowed, the last rule cannot be used without adjustment: $n + (-n) = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

little-o works for every positive multiplier

$$f \in o(g) \iff \forall c > 0\ \exists n_0(c)\ \forall n \geq n_0(c) :\ 0 \leq f(n) < cg(n).$$

Big-O asks for one multiplier; little-o asks for **every positive multiplier**. For $7n \in o(n^2)$, given $c > 0$, choose an integer $n_0 > 7/c$.

Likewise $f \in \omega(g)$ requires $0 \leq cg(n) < f(n)$ eventually for every $c > 0$. Hence $n^2 \in \omega(7n)$.

For eventually positive functions, little-o is equivalent to $f/g \to 0$, and little-omega to $f/g \to \infty$. $7n^2$ and n^2 have the same order; their ratio does not tend to zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

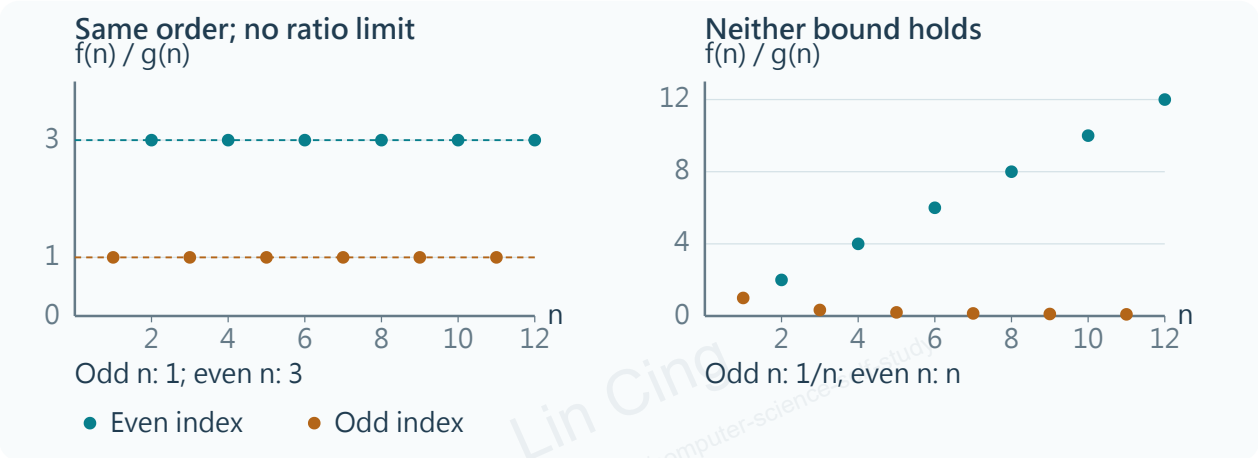
A ratio limit is a tool, not the only criterion

$\lim f(n)/g(n)$	Conclusion
0	$f \in o(g)$
Finite positive L	$f \in \Theta(g)$
$+\infty$	$f \in \omega(g)$
Does not exist	Continue the analysis; incomparability does not follow

For finite positive L, choose $\varepsilon = L/2$. Eventually $L/2 < f/g < 3L/2$, providing two fixed positive multipliers.

The first three rows require the stated limit to exist. Logarithmic comparison also requires positive arguments: $\lg f / \lg g \to 1$ alone does **not** imply $f = \Theta(g)$.

An oscillating ratio can still be tightly bounded



Left: $f = n(2 + (-1)^n), g = n$ satisfy $n \le f \le 3n$ despite oscillation. Right: $g = n$; use $f = n^2$ for even n and $f = 1$ for odd n. Even ratios are unbounded, refuting $O(g)$; odd ratios tend to zero, refuting $\Omega(g)$.

Do not exchange function bounds and input quantifiers

Let $C(x)$ be input cost and $W(n) = \max_{x \in D_n} C(x)$. If $W(n) \in O(n^2)$, there are **shared** constants c, n_0 such that every $n \geq n_0$ and every $x \in D_n$ satisfy $C(x) \leq cn^2$.

If a separate constant may grow with each input, the statement does not provide that uniform upper bound.

Insertion sort has $W(n) = \Theta(n^2)$, yet a sorted input takes $\Theta(n)$. A tight worst-case bound does not say that every input requires quadratic time.

Three practical approaches to comparing growth

Approach	Useful for	Common mistake
Definition and inequalities	Polynomials, oscillation, explicit constants	Only sampling values or proving one side
Ratio limit	Positive functions with a manageable limit	Treating nonexistence as incomparability
Take logarithms	Positive products, powers, exponentials	Inferring original tight bounds from tight logarithms

For $f = n \lg n, g = n$, $\lg f / \lg g = 1 + \lg\lg n / \lg n \to 1$, but $f/g = \lg n \to \infty$. Thus $f \in \omega(g)$.

Logs can simplify comparison. A fixed multiplicative bound still requires the original ratio, or a sufficiently precise bound on the difference of logarithms.

Growth hierarchies require fixed constants

For fixed $a > 1, b > 0, \varepsilon > 0$, and $k > 0$,

$$1 \prec \lg\lg n \prec (\lg n)^b \prec n^\varepsilon \prec a^n \prec n! \prec n^n.$$

Here $f \prec g$ means $f = o(g)$, for sufficiently large positive integers.

With $x = \ln n$, $(\lg n)^b / n^\varepsilon$ is a fixed constant times $x^b / e^{\varepsilon x}$, which tends to zero. Also $\ln(n^k / a^n) = k \ln n - n \ln a \to -\infty$, proving $n^k = o(a^n)$.

The following panel justifies the basic limit $\ln n / n \to 0$. If a base or exponent varies with n, these fixed-constant conclusions need not apply.

Why does a positive power dominate a logarithm?

For real $x \geq 4$, the derivative of $\sqrt{x} - \ln x$ is $(\sqrt{x} - 2)/(2x) \geq 0$, and its value at 4 is positive. Hence $0 \leq \ln x / x \leq 1/\sqrt{x} \to 0$.

Now fix $b, \varepsilon > 0$ and set $t = \ln n$. Taking natural logarithms gives

$$\ln \frac{(\ln n)^b}{n^\varepsilon} = b \ln t - \varepsilon t = t \left(b \frac{\ln t}{t} - \varepsilon \right).$$

Since $\ln t / t \to 0$, the parenthesis is eventually at most $-\varepsilon/2$. The logarithm tends to $-\infty$, so the original ratio tends to zero. Thus $(\ln n)^b = o(n^\varepsilon)$; a fixed change of log base changes only a constant.

Logarithms, powers, and changes of base

Expression	Interpretation
$\log_a n = \lg n / \lg a$	Fixed $a > 1$: log bases differ by a constant
$\lg(n^k) = k \lg n$	Take a power, then a logarithm
$\lg^k n = (\lg n)^k$	Take a logarithm, then a power
$2^{\lg n} = n$	Matching exponential and logarithm bases
$2^{2 \lg n} = n^2$	A constant in an exponent cannot simply be discarded

$\lg n$ and $2 \lg n$ have the same order; $2^{\lg n}$ and $2^{2 \lg n}$ do not. A varying log base is another issue: $\log_n n = 1$ does not behave like a fixed-base logarithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From a loop to a sum: recheck each iteration's range

```
count = 0
for i = 1 to n
    for j = 1 to i
        count = count + 1
```

Iteration i performs i increments, so

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Saying the inner loop averages about half the maximum is not a proof. The exact sum supplies a formula. Alternatively, n^2 is an upper bound, while the final half of the iterations each perform at least $n/2$ operations, giving a lower bound $n^2/4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A triangular iteration space is still quadratic

n = 6; one cell is one visit(i, j)



Upper: at most 6 × 6 = 36 cells. Lower: three cells in each of rounds 4–6.

Each cell is one inner operation. For $n = 6$, $1 + 2 + 3 + 4 + 5 + 6 = 21$ cells. Taking three cells from each of rows 4–6 gives $3 \times 3 = 9$: a concrete instance of the preceding general quadratic lower bound.

A doubling loop counts how often 2 is multiplied

```
i = 1
while i <= n
    visit(i)
    i = 2*i
```

With unbounded integer arithmetic and $n \geq 1$, it visits $1, 2, 4, \ldots, 2^{\lfloor \lg n \rfloor}$, exactly $\lfloor \lg n \rfloor + 1$ values. For example, input 13 visits $1, 2, 4, 8$: four values.

A fixed-width implementation must avoid overflow in `2*i`. After visiting, stop if $i > \lfloor n/2 \rfloor$; otherwise doubling is safe. A mathematical termination proof does not automatically cover machine overflow.

A geometric sum has the order of its last term

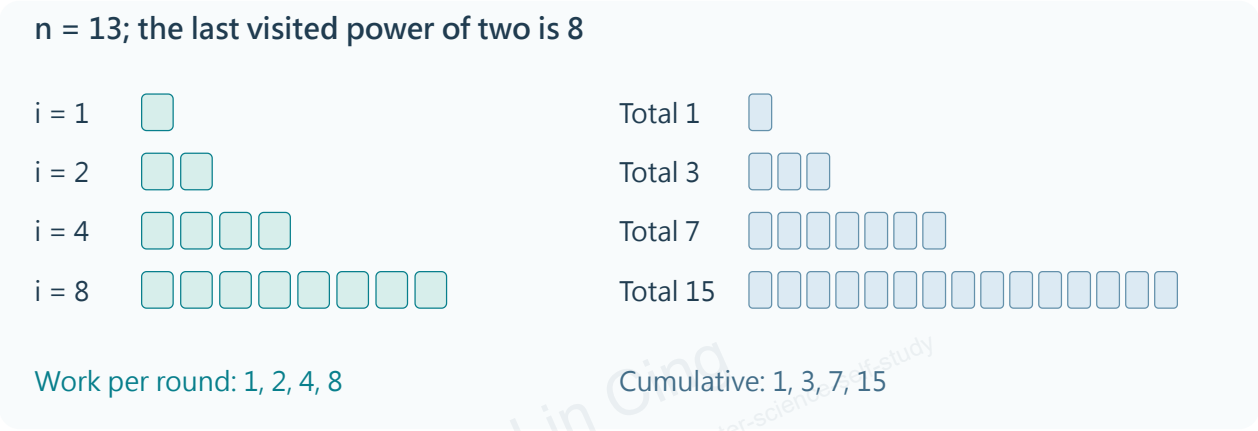
If the outer values are $i = 1, 2, 4, \ldots, 2^k \leq n$ and each iteration performs i operations, the total is

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

Since $2^k \leq n < 2^{k+1}$, we have $n/2 < 2^k \leq n$. Thus the total is $\Theta(n)$, not $\Theta(n \lg n)$.

For $n = 13$, the four iterations perform $1, 2, 4, 8$ operations, totaling 15. Replacing every iteration's work by n gives an upper bound, but it does not establish a tight bound.

Sum the changing work, not the final work times the rounds



Rounds cost $1, 2, 4, 8$; cumulative work is $1, 3, 7, 15$. Generally, $2^k \le \sum_{r=0}^{k} 2^r < 2 \cdot 2^k$. The total is within twice the last round, with no extra factor for the number of rounds.

Bound a harmonic sum by areas on both sides

Let $H_n = \sum_{i=1}^{n} 1/i$. The function $1/x$ decreases on the positive real axis, so

$$\int_1^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_1^{n} \frac{dx}{x}.$$

Thus $\ln(n+1) \le H_n \le 1 + \ln n$. For $n \ge 3$, $1 + \ln n \le 2 \ln n$, giving $H_n = \Theta(\ln n) = \Theta(\lg n)$.

For the upper bound, separate the first term 1 and place later terms below the curve on the preceding intervals. For the lower bound, left-endpoint rectangles cover the integral. The following panel separates the two constructions.

Power sums need not have a closed form to be bounded

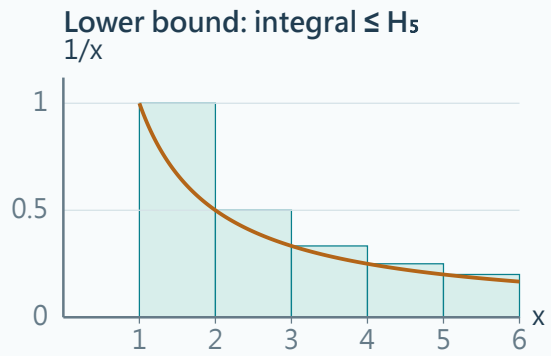
For fixed real $p \ge 0$, let $S_p(n) = \sum_{i=1}^{n} i^p$. There are n terms, each at most n^p, so $S_p(n) \le n^{p+1}$.

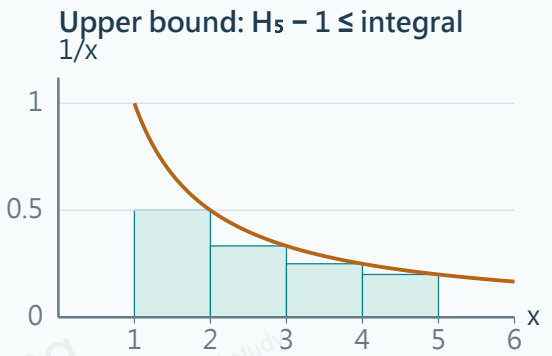
For a lower bound, retain $i = \lceil n/2 \rceil .. n$. There are at least $n/2$ such terms, each at least $(n/2)^p$, so

$$S_p(n) \ge \frac{n}{2} \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}.$$

Thus $S_p(n) = \Theta(n^{p+1})$. The factors are fixed because p is fixed. The case $p = -1$ is harmonic and does not satisfy this conclusion.

One curve, two rectangle constructions





Left: height $1/i$ on $[i, i+1]$ covers the curve. Right: terms $i = 2..5$ lie below it on $[i-1, i]$. The common-scale panels give $1.7918 \approx \ln 6 \leq H_5 = 137/60 \approx 2.2833 \leq 1 + \ln 5 \approx 2.6094$.

Dependent nested loops can take n log n time

```
for i = 1 to n
    j = i
    while j <= n
        visit(i, j)
        j = j + i
```

The inner loop visits $i, 2i, \ldots, \lfloor n/i \rfloor i$, giving

$$S(n) = \sum_{i=1}^{n} \lfloor n/i \rfloor .$$

For $x \geq 1$, $x/2 \leq \lfloor x \rfloor \leq x$. Hence $nH_n/2 \leq S(n) \leq nH_n$, proving $\Theta(n \lg n)$. At $n = 6$, the counts are $6, 3, 2, 1, 1, 1$, totaling 14.

Take logarithms before comparing factorial growth

For $n \geq 4$, $n! \leq n^n$, so $\lg(n!) \leq n \lg n$.

At least $n/2$ factors in the latter half are at least $n/2$, giving

$$\lg(n!) \geq \frac{n}{2} \lg(n/2) \geq \frac{n}{4} \lg n.$$

Hence $\lg(n!) = \Theta(n \lg n)$. This does not imply $n! = \Theta(n^n)$: that ratio tends to zero.

For fixed $a > 1$, $\ln(n!/a^n) \geq (n/2)\ln(n/2) - n \ln a \to \infty$, proving $a^n = o(n!)$. Use Stirling's formula for a more precise factor; an error inside a logarithm cannot be exponentiated and discarded indiscriminately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries for floor, ceiling, and remainder

For real x, $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$. For integer n, this implies

$$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n.$$

For integer a and positive integer m, use the mathematical remainder

$$a \bmod m = a - m\lfloor a/m \rfloor, \qquad 0 \leq a \bmod m < m.$$

For example, $-7 \bmod 3 = 2$, since $-7 = 3(-3) + 2$. A language's `%` operator may instead use a quotient truncated toward zero and produce a negative remainder. Check the semantics before translating a formula into code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When can successive integer divisions be combined?

For $x \geq 0$ and positive integers a, b,

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \qquad \left\lceil \frac{\lceil x/a \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil.$$

For floor, set $q = \lfloor x/(ab) \rfloor$, so $abq \leq x < ab(q+1)$. Divide by a and take floor: $bq \leq \lfloor x/a \rfloor \leq bq + b - 1$. Dividing by b and taking floor now gives q.

For $x = 17, a = 3, b = 2$, successive floors give $17 \to 5 \to 2$, matching $\lfloor 17/6 \rfloor = 2$. Positivity and integrality of the divisors support the integer-interval argument.

The ceiling proof uses a strict lower endpoint

Set $q = \lceil x/(ab) \rceil$. Then $ab(q-1) < x \leq abq$. Divide by a and take ceiling:

$$b(q-1) + 1 \leq \lceil x/a \rceil \leq bq.$$

Dividing by b and taking ceiling gives q. For $x = 17, a = 3, b = 2$, the steps $17 \to 6 \to 3$ match $\lceil 17/6 \rceil = 3$.

For positive integers u, v, $\lceil u/v \rceil \leq (u + v - 1)/v$ and $\lfloor u/v \rfloor \geq (u - v + 1)/v$. Write $u = qv + r$, $0 \leq r < v$, and check $r = 0$ and $r > 0$ separately. The implementation `(u+v-1)/v` still needs an overflow check for the addition.

Peak space counts what is alive simultaneously

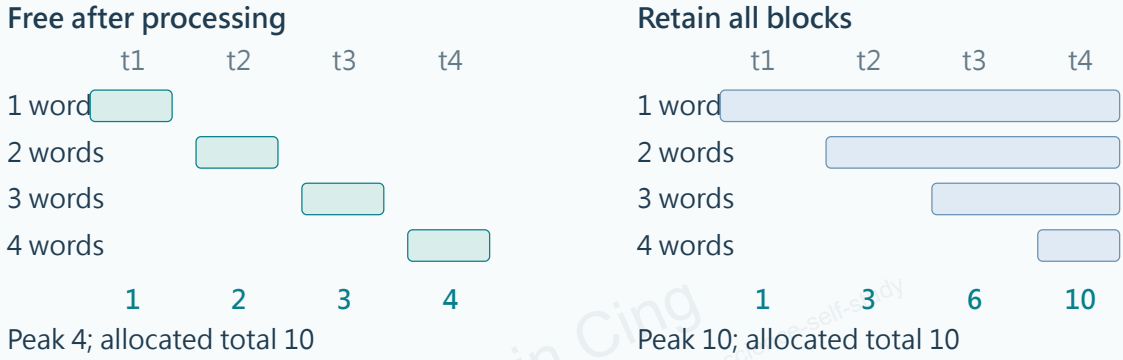
Auxiliary space excludes the input but normally includes temporary arrays and the call stack. If output space is also excluded, state that convention.

Situation	Cumulative allocation	Peak live storage
Allocate and free n single-word blocks in turn	$\Theta(n)$	$\Theta(1)$
Retain all n single-word blocks	$\Theta(n)$	$\Theta(n)$
K calls; peak of d constant-size live frames	$\Theta(K)$	$\Theta(d)$

Insertion sort keeps a constant number of indices and temporary records. Its auxiliary space is $\Theta(1)$ when each record occupies a constant number of words. For large objects, clarify whether the algorithm moves objects or references.

Draw allocation lifetimes to find the peak

Allocate 1, 2, 3, 4 words; horizontal positions are processing intervals



Allocate blocks of 1, 2, 3, and 4 words. Immediate release gives peak 4; retaining all gives peak 10. Both cumulatively allocate ten words. Count these live blocks only, excluding allocator fragmentation, caches, and operating-system resident memory.

Constants and crossover points still matter

In the same cost unit, let $T_A(n) = 8n^2$ and $T_B(n) = 64n \lg n$, $n \geq 2$. A is faster exactly when $n < 8 \lg n$.

n	$8n^2$	$64n \lg n$	Comparison
16	2048	4096	A faster
32	8192	10240	A faster
64	32768	24576	B faster

For real $n \geq 4$, $n/\lg n$ is strictly increasing. Checking 43 and 44 on opposite sides, together with 2 and 3, locates the integer crossover between 43 and 44. If these formulas came from measurements, the result would be conditional on that model and environment.

Hardware speed has units too

If A executes 10^9 model operations per second and B executes 10^8, compare

$$t_A(n) = T_A(n)/10^9, \qquad t_B(n) = T_B(n)/10^8.$$

Do not compare an operation count directly with a time in seconds. Hardware changes constants; an algorithmic change may change the growth order. Both comparisons need an explicit model.

Also check space: an n^2-entry table may exceed memory even when its running time is acceptable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Put the analysis together

For a new program, build the argument in this order rather than guessing a Big-O first:

1. Fix legal inputs, input size, and the comparison/arithmetic model.
2. Trace a small input completely to identify what each iteration does.
3. State invariants, boundary cases, and a decreasing quantity.
4. Write a general sum or recurrence for the cost.
5. Establish upper and lower bounds; identify the best/worst/average function.
6. Find peak space and check overflow, aliasing, and distribution assumptions.

For insertion sort, $T(S) = \Theta(n + I(S))$ explains how input structure affects work, beyond the looser statement of worst-case $O(n^2)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: why does a polynomial's leading term dominate?

Let $p(n) = a_d n^d + \sum_{i<d} a_i n^i$, with fixed $d \geq 1$, $a_d > 0$, and $M = \sum_{i<d} |a_i|$.

For $n \geq 1$, the lower terms have total absolute value at most Mn^{d-1}. If also $n \geq 2M/a_d$, then

$$\frac{a_d}{2} n^d \leq p(n) \leq \frac{3a_d}{2} n^d.$$

Thus $p(n) = \Theta(n^d)$ even with negative lower coefficients. A negative leading coefficient violates our eventually nonnegative cost convention. If $d = 0, a_0 > 0$, the result is $\Theta(1)$ instead.

Extension: an asymptotic equality need not be reversible

The expression $n^2 + n = n^2 + O(n)$ means that a function in $O(n)$ makes the equality true; here it is the function n.

It implies $n^2 + n = \Theta(n^2)$, but $f = \Theta(n^2)$ does not imply $f = n^2 + O(n)$. Take $f = 2n^2$: the difference n^2 is not in $O(n)$.

To calculate with anonymous functions, name them first. For $u, v \in O(n)$, the triangle inequality gives $|u + v| \leq (c_u + c_v)n$ eventually. Subtraction requires a convention for signed error bounds; do not silently ignore the nonnegative-function convention used in a definition.

Extension: e, ln, and approximation error

For $x > -1$, $\ln(1 + x) \leq x$. Therefore $(1 + 1/n)^n \leq e$ for positive integers n.

For $x \geq 0$, a more precise pair of bounds is

$$x - \frac{x^2}{2} \leq \ln(1 + x) \leq x.$$

For the lower bound, set $h(x) = \ln(1 + x) - x + x^2/2$. Then $h(0) = 0$ and $h'(x) = x^2/(1 + x) \geq 0$. Substituting $x = 1/n$ gives

$$e^{1-1/(2n)} \leq (1 + 1/n)^n \leq e.$$

This bounds finite-n error, not just the limit. Approximations require a domain and an error bound; higher-order terms cannot always be discarded.

Extension: an exponential remainder with explicit bounds

Taylor's theorem gives $e^x = 1 + x + (e^\xi/2)x^2$, with ξ between 0 and x. For $|x| \le 1$,

$$\frac{e^{-1}}{2}x^2 \le e^x - 1 - x \le \frac{e}{2}x^2.$$

Thus $e^x = 1 + x + \Theta(x^2)$ as $x \to 0$, rather than as $x \to \infty$. Convexity gives $e^x \ge 1 + x$ for every real x.

The standard power series $e^x = \sum_{k=0}^{\infty} x^k/k!$ converges for every real x. Using only a few terms requires a remainder estimate such as the one above. This chapter uses the basic calculus theorem, rather than deriving Taylor's theorem itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: logarithmic bounds also work for negative x

For $x > -1$, the general pair is

$$\frac{x}{1+x} \le \ln(1+x) \le x.$$

For the upper bound, $h = x - \ln(1+x)$ has derivative $x/(1+x)$ and its minimum at 0 is zero. For the lower bound, $q = \ln(1+x) - x/(1+x)$ has derivative $x/(1+x)^2$ and the same minimum. Both arguments cover negative and positive x.

For fixed real z, substitute $x = z/n$ for sufficiently large n: $z/(1+z/n) \le n\ln(1+z/n) \le z$. Both sides tend to z, proving $(1+z/n)^n \to e^z$.

For $|x| < 1$, the standard series is $\ln(1+x) = \sum_{k=1}^{\infty}(-1)^{k+1}x^k/k$; its stated domain matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: how much precision does Stirling supply?

For positive integers n, Stirling's formula is

$$n! = \sqrt{2\pi n}(n/e)^n\big(1 + O(1/n)\big).$$

Stirling's theorem controls the relative error of the leading approximation. Taking natural logarithms gives

$$\ln(n!) = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + O(1/n).$$

The relative error $O(1/n)$ becomes a logarithmic error $O(1/n)$ since $\ln(1+u) = O(u)$ for sufficiently small u. Conversely, a difference $O(n)$ between logarithms can become an exponential factor, not a constant relative error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: turn Stirling's error into a numerical interval

A sharper form of Stirling's theorem is

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \quad (n \geq 1).$$

For $n = 6$, the leading approximation $L = \sqrt{12\pi}(6/e)^6 \approx 710.078$ is below $6! = 720$. The bounds give

$$Le^{1/73} < 720 < Le^{1/72}.$$

The correction is bounded, not an arbitrary rounding choice. Since $\alpha_n = \Theta(1/n)$ and $e^u - 1 = \Theta(u)$ near zero, $e^{\alpha_n} = 1 + \Theta(1/n)$. Coarse product bounds establish the order of $\lg(n!)$; this theorem additionally controls the leading approximation's relative error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: iterated logs need a stopping rule

Set $\lg^{(0)} n = n$ and $\lg^{(k+1)} n = \lg(\lg^{(k)} n)$, taking another log only when the previous value exceeds 1. Define

$$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

Input	Successive values	$\lg^* n$
1	1	0
2	$2 \to 1$	1
16	$16 \to 4 \to 2 \to 1$	3
65536	$65536 \to 16 \to 4 \to 2 \to 1$	4

$\lg^k n$ raises a logarithm to a power; $\lg^{(k)} n$ repeatedly applies a function. The notation and growth behavior differ.

Extension: verify the Fibonacci closed form

Define $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Set $\varphi = (1 + \sqrt{5})/2$ and $\psi = (1 - \sqrt{5})/2$; both satisfy $r^2 = r + 1$.

Therefore $G_n = (\varphi^n - \psi^n)/\sqrt{5}$ satisfies the same recurrence, with $G_0 = 0, G_1 = 1$. Induction gives $G_n = F_n$ for every nonnegative integer.

Since $|\psi| < 1 < \varphi$, $F_n = \Theta(\varphi^n)$. But the **size of the answer** is not the time needed to compute it. Naive recursion, a table, and fast doubling require separate analyses in the recurrence chapter.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study